

CSI 2110
Data Structures and Algorithms

Lecture 3: Lists

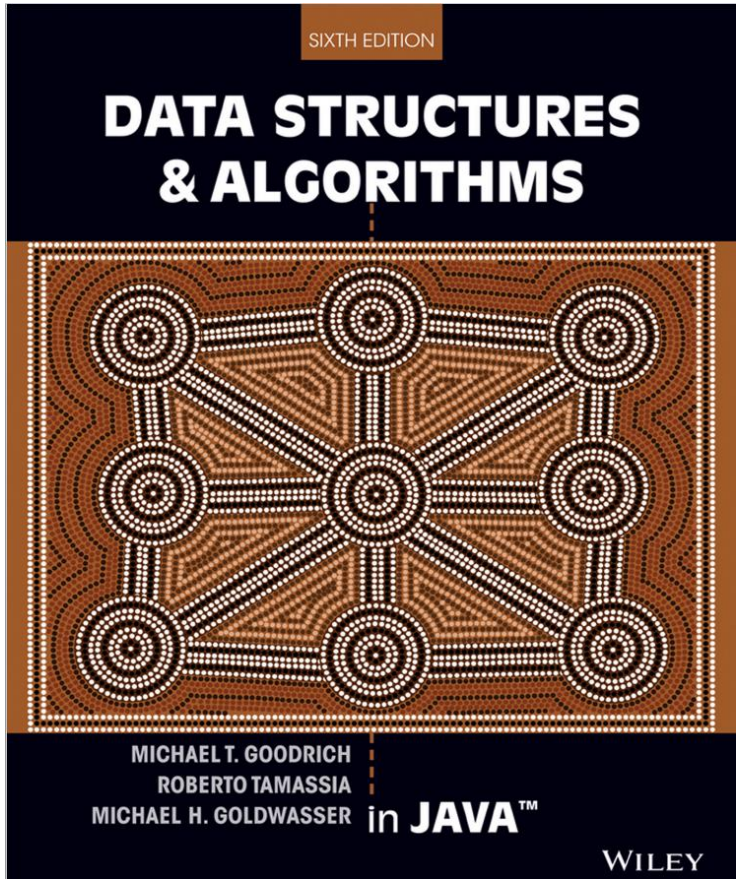
Bo Sun

Fall 2026

A note on the slides

Adapted from the slides that accompany this book.

Part of the slides is based on previous slides from Prof. Lucia Moura for CSI 2110.

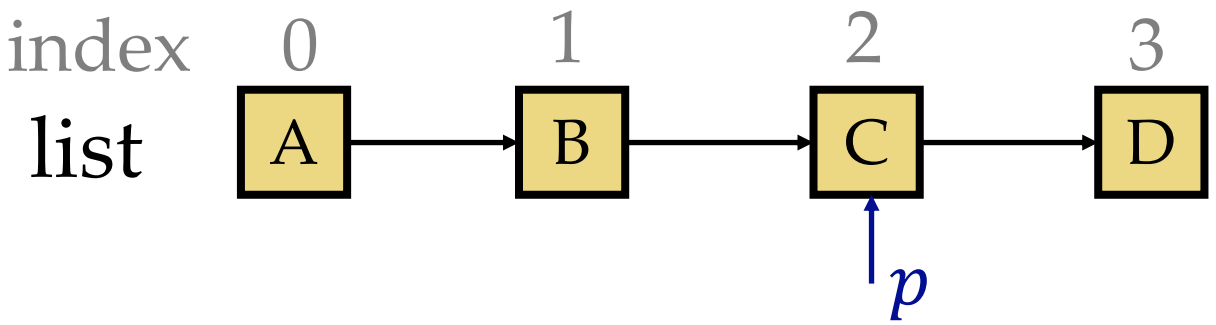


Data Structures and Algorithms in Java
6th edition
Michael Goodrich, Roberto Tamassia, Michael Goldwasser
Wiley, 2014

Beyond operations at the ends of a list

- A deque supports insertion and removal at both ends

  What if we need to insert or delete elements in the middle of a list?

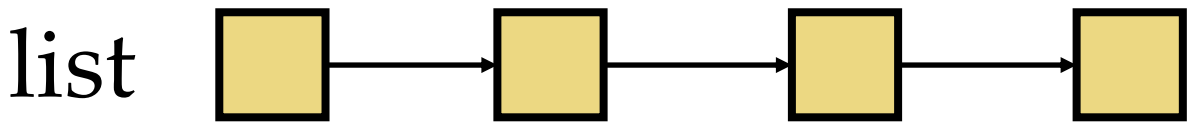


- By index
  - Insert X at index 2
  - Array-list ADT
- By position
  - Insert X before position p
  - Positional-list ADT

Same insertion, two ways to identify where it happens

Lists

- List is a collection of elements in linear order



- Array List
  - accessed by index
  - to be implemented by arrays
- Positional List
  - accessed by position (or address)
  - to be implemented by linked list
- Sequence
  - Combination of array list and positional list

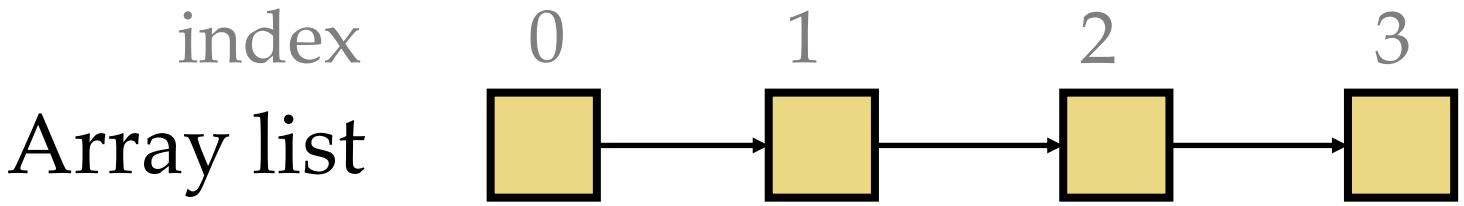
Lists: roadmap

- Array list
  - Extendable array
- Positional list
- Sequence

Array list

- An array list is a collection of elements in linear order that can be accessed by index (or rank)
  - index of an element is the number of elements that precede it

- Array list is typically implemented by an array



Array list ADT

Main methods for an array list with n elements

- get(i): return the element with index i (or an error if $i < 0$ or $i > n - 1$)
- set(i, e): replace the element at index i with e, and return the old element (or an error if $i < 0$ or $i > n - 1$)
- add(i, e): insert a new element that will have index i (or an error if $i < 0$ or $i > n$)
- remove(i): remove the element at index i (or an error if $i < 0$ or $i > n - 1$)

Support methods

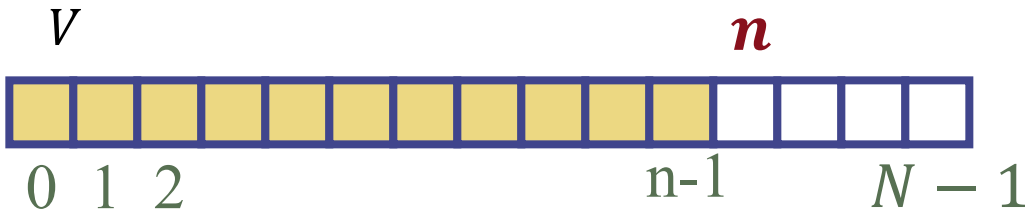
- size(): return the number of elements of the list
- isEmpty(): return a Boolean indicating whether the list is empty

Example



Implementing an array list with an array

- The array-list consists of an N-element array V and a size variable n (number of elements stored)




```
V ← new array of N elements
n ← 0      ▷ empty array list
Algorithm get(i)
    return V[i]
Algorithm set(i,e)
    temp ← V[i]
    V[i] ← e
    return temp
```

Insertion

- To insert an element at index i, we need to make room for the new element by shifting forward the $n - i$ elements $V[i], \ldots, V[n-1]$
- In the worst-case ($i = 0$), it takes $O(n)$ time

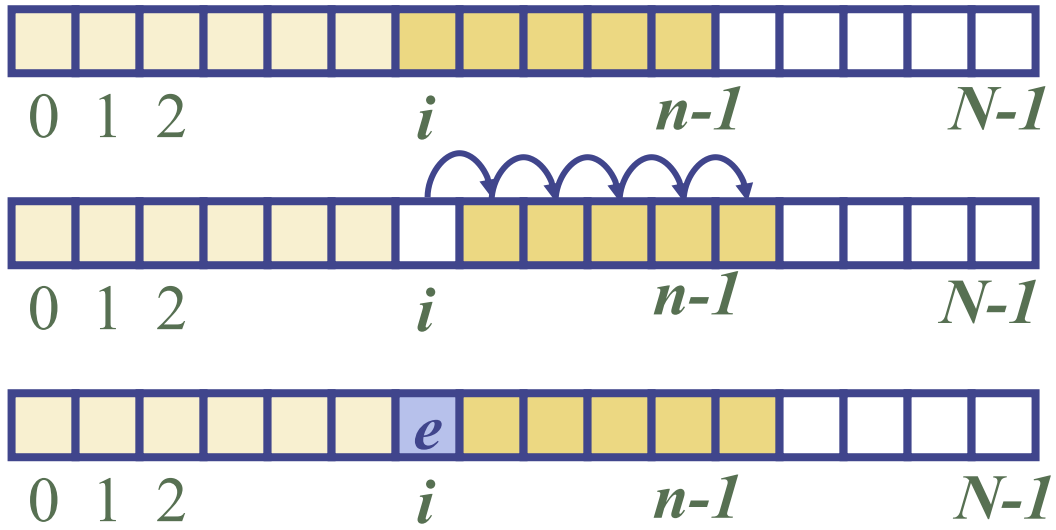
Algorithm add(i,e)

For $j \leftarrow n-1$ **downto** i **do**

$V[j+1] \leftarrow V[j]$

$V[i] \leftarrow e$

$n \leftarrow n+1$



Deletion

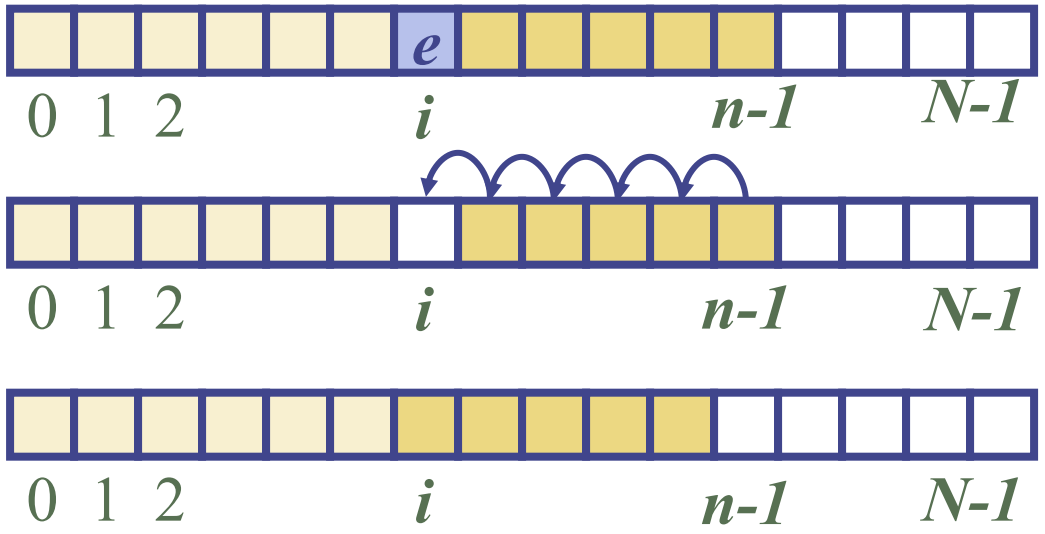
- After deleting an element at index i, we need to fill the hole by shifting backward the $n - i - 1$ elements $V[i + 1], \ldots, V[n - 1]$
- In the worst-case ($i = 0$), it takes $O(n)$ time

Algorithm `remove(i,e)`

For $j \leftarrow i$ **to** $n - 2$ **do**

$V[j] \leftarrow V[j + 1]$

$n \leftarrow n - 1$



Array-list with arrays: Performance

- Worst-case time and space

Time:

size()	$O(1)$
isEmpty()	$O(1)$
get(i)	$O(1)$
set(i,e)	$O(1)$
add(i,e)	$O(n)$
remove(i)	$O(n)$

Space: $O(N)$

N = size of the array

- When adding an element, if the array is full, instead of having an error, we can replace the array with a large one: extendable array

Lists: roadmap

- Array list
  - Extendable array
- Positional list
- Sequence

Extendable array list

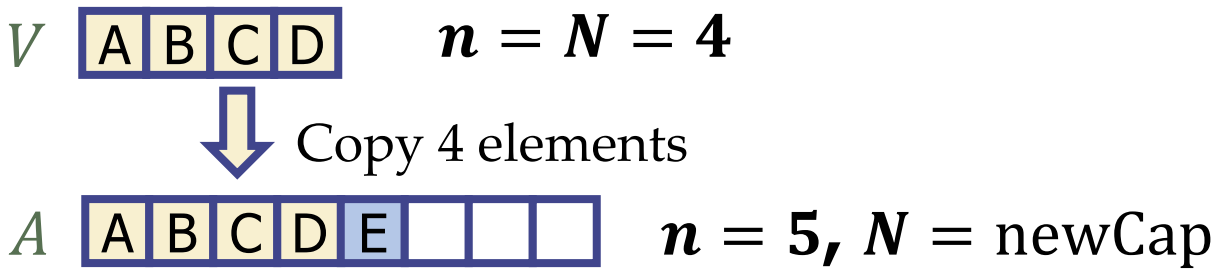
- append(e) adds element e at the end of the list
- When the array is full, replace the array with a larger one

  n = current number of elements; N = current array capacity

Algorithm append(e)

If $n = N$ **then**
  choose newCap $> N$
  $A \leftarrow$ new array of size newCap
  For $j \leftarrow 0$ **to** $n - 1$ **do**
    $A[j] \leftarrow V[j]$
  $V \leftarrow A$
  $N \leftarrow$ newCap
$V[n] \leftarrow e$
$n \leftarrow n + 1$

Example: append(E)



Amortized analysis

- Start with an empty array list with capacity 1
- Let $T(m)$ be the total time for m append(e)

Amortized time per operation = $T(m)/m$

Incremental strategy

- When array is full, increase capacity by a fixed positive integer c

$$\text{newCap} \leftarrow N + c$$

Example: $c = 2$

Operations number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Capacity	1	3	3	5	5	7	7	9	9
Elements copied	0	1	0	3	0	5	0	7	0

$$\#\text{of copies} = 1 + 3 + \cdots + (2k - 1)$$
$$= k^2 < \frac{(m+1)^2}{2} \in O(m^2)$$

where k is the number of capacity updates and $2k - 1 < m$

Total time: $T(m) \in O(m^2)$

Amortized time: $T(m)/m \in O(m)$

Any fixed c > 0 gives the same asymptotic bound

Doubling strategy

- When array is full, increase capacity by doubling its current capacity

$$\text{newCap} \leftarrow 2N$$

Example:

Operations number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Capacity	1	2	4	4	8	8	8	8	16
Elements copied	0	1	2	0	4	0	0	0	8

$$\#\text{of copies} = 1 + 2 + 2^2 + \cdots + 2^k$$

where k is the number of capacity updates and $2^k < m$ ($k < \log m$)

$$= 2^{k+1} - 1 < 2 \times 2^{\log m} - 1 = 2m - 1 \in O(m)$$

Total time: $T(m) \in O(m)$

Amortized time: $T(m)/m \in O(1)$

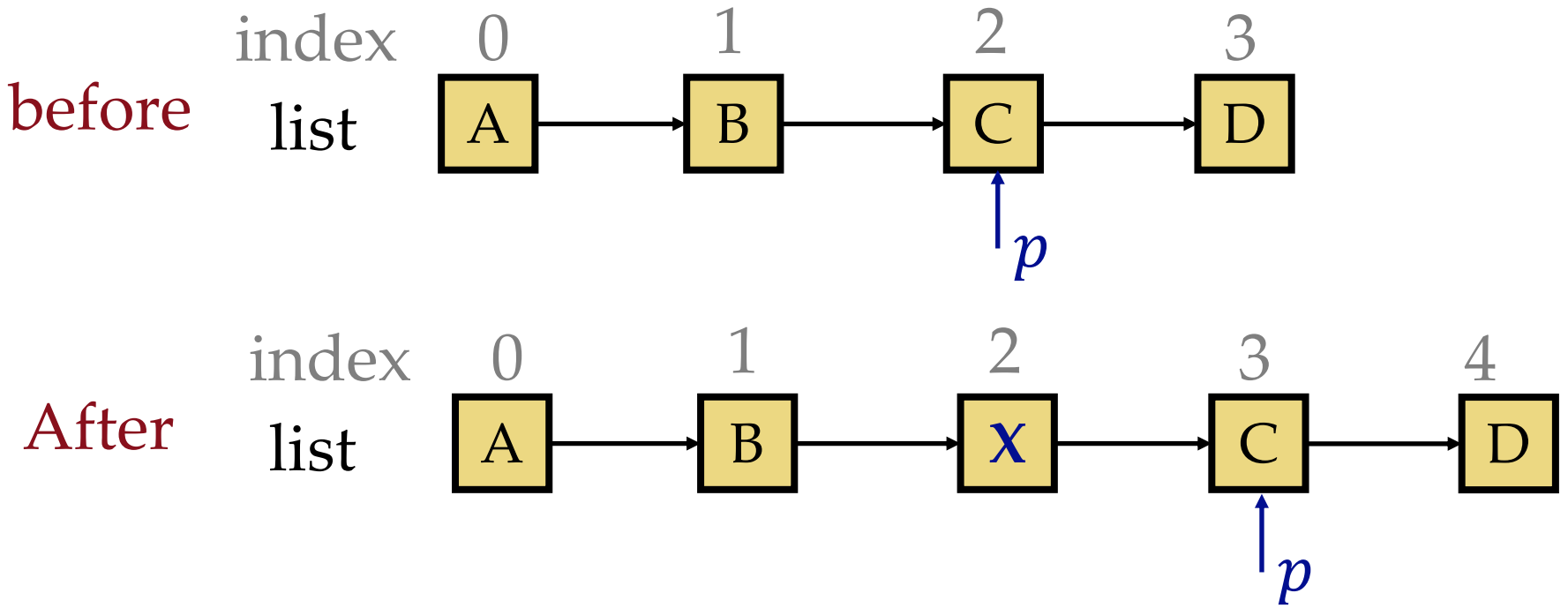
Lists: roadmap

- Array list
  - Extendable array
- Positional list
- Sequence

What is a position?

- A position is a reference (handle) to a particular entry (e.g., object reference)

- Consider inserting X before position p

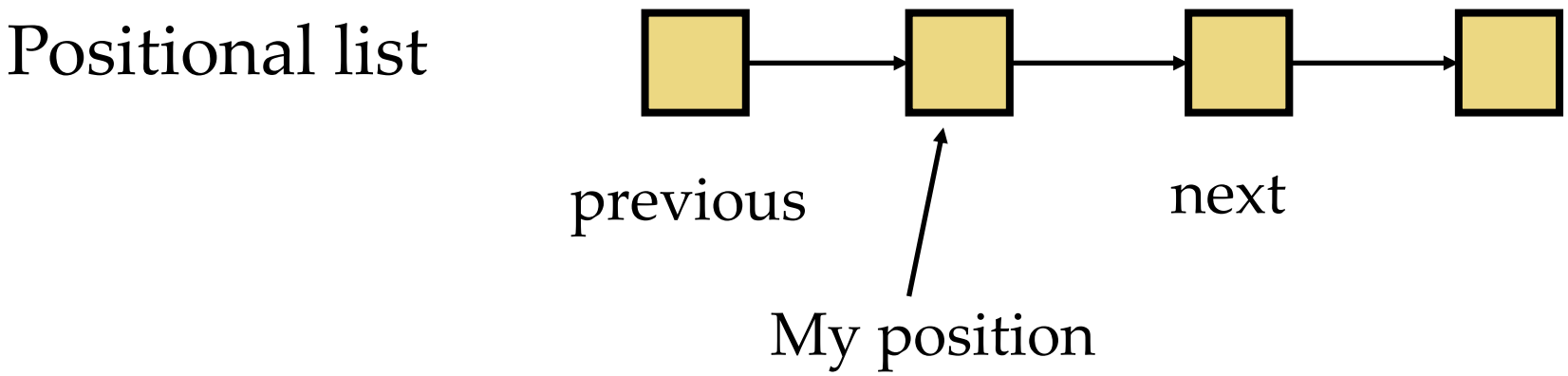


The entry containing C moves from index 2 to index 3, but position p still refers to the same entry

Positional list

- A positional list is a collection of positions in linear order, where each position stores an element.
  - positions are defined relative to other positions (before or after)

- Positional list does not provide index-based access.
  - Given a position p, we can access the positions immediately before or after p



Positional list ADT

Accessor methods

- first()/last(): return the position of the first/last element
- before(p)/after(p): return the position immediately before/after position p

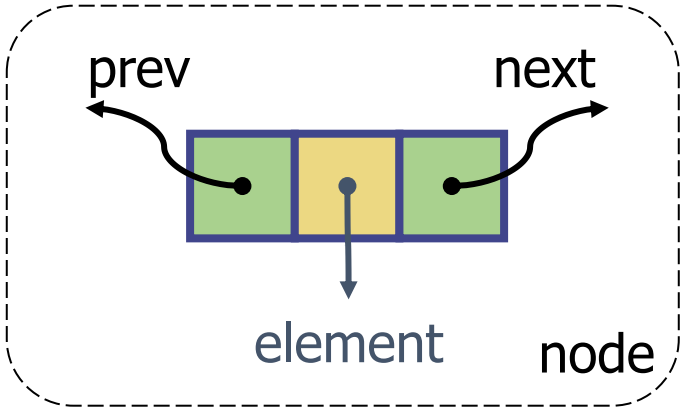
Update methods

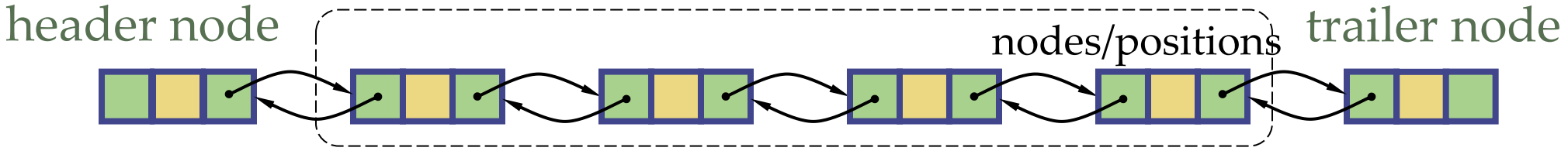
- addFirst(e)/addLast(e): insert element e at the front/back of the list and return the position of the new element
- addBefore(p, e)/addAfter(p,e): insert element e, just before/after position p, and return the position of the new element
- set(p, e): replace the element at position p and return the element formerly at position p
- remove(p): remove and return the element at position p, and invalidate the position

Support methods: size(), isEmpty()

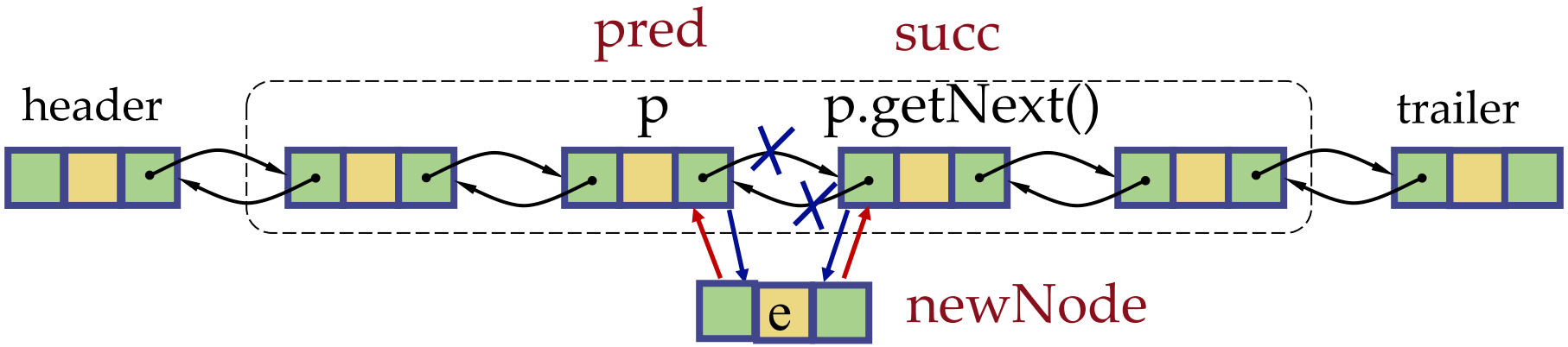
Implementing a positional list with a linked list

- A doubly linked list provides a natural implementation of the positional list
- Nodes implement positions and store
  - element
  - link to the previous node
  - link to the next node





Insertion: addAfter(p, e), addLast(e)



Algorithm addBetween(e,pred,succ):

newNode ← new Node;
newNode.setElement(e);
newNode.setPrev(pred);
newNode.setNext(succ);
pred.setNext(newNode);
succ.setPrev(newNode);
$sz \leftarrow sz + 1$
return newNode

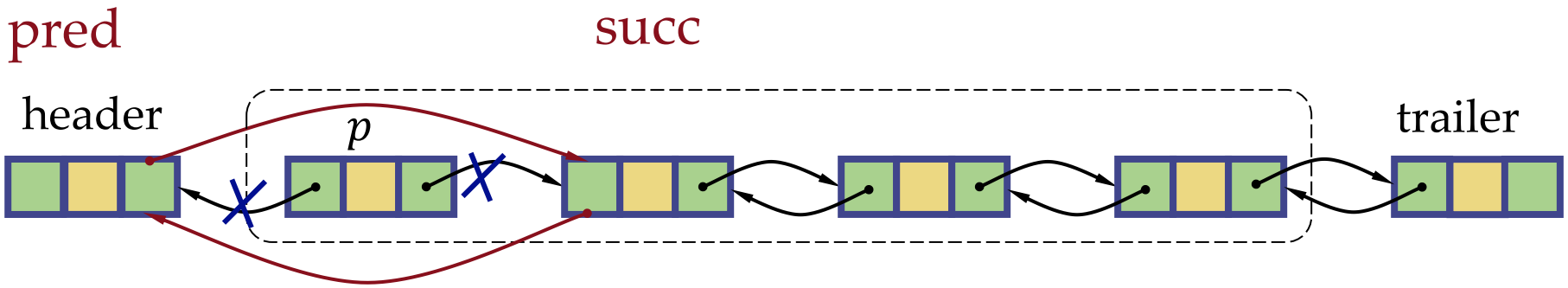
Algorithm addAfter(p,e):

Return addBetween(e, p, p.getNext());

Algorithm addLast(e):

Return addBetween(e, tailer.getPrev(), tailer)

Deletion: remove(p)




```
Algorithm remove(p):
  pred ← p.getPrev();
  succ ← p.getNext();
  pred.setNext(succ);
  succ.setPrev(pred);
  sz ← sz − 1
  return p.getElement()
```

Positional list with linked lists: Performance

- Worst-case time and space efficiency

Time:

size()	$O(1)$
isEmpty()	$O(1)$
addFirst(e)	$O(1)$
addLast(e)	$O(1)$
addBefore(p,e)	$O(1)$
addAfter(p,e)	$O(1)$
set(p,e)	$O(1)$
remove(p)	$O(1)$

Assume a valid position is given.

Space: $O(n)$

n = number of the elements in the list

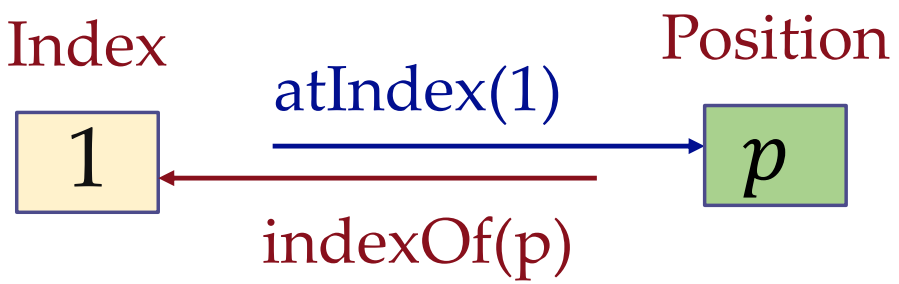
Lists: roadmap

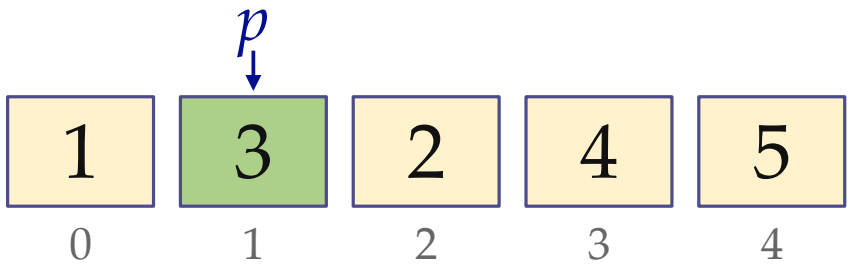
- Array list
  - Extendable array
- Positional list
- Sequence

Sequence ADT: access by index and position

- A sequence ADT combines array-list ADT and Positional-list ADT and provides all of their methods plus bridge methods:

- Bridge methods between index and positions:
  - atIndex(i): return a position of index i
  - indexOf(p): return the index of position p





Move-to-front encoding

- Input: an initial ordered list $S = [1,2,\ldots,m]$, and an input sequence Y of length n, where $Y[i] \in \{1,2,\ldots,m\}$
- For each input value $Y[i]$, record its current index in $Z[i]$, then move it to the front of S
- Output: the index sequence Z

L=[1,2,3,4,5], Y=[3,4,3,1]

Obtain the current index
indexOf(p)

Query $Y[i]$	List L before query	Index Z[i]	List L after move-to-front
3	[1, 2, 3, 4, 5]	2	[3, 1, 2, 4, 5]
4	[3, 1, 2, 4, 5]	3	[4, 3, 1, 2, 5]
3	[4, 3, 1, 2, 5]	1	[3, 4, 1, 2, 5]
1	[3, 4, 1, 2, 5]	2	[1, 3, 4, 2, 5]

Move to front:
remove(p) and
addFirst(e)

Move-to-front encoding with a sequence

- Maintain an auxiliary array loc, where loc[x] stores the position containing value x.

Initialization

```
S ← empty sequence
loc ← new array[m + 1]
for x ← 1 to m do
    loc[x] ← S.addLast(x)
```

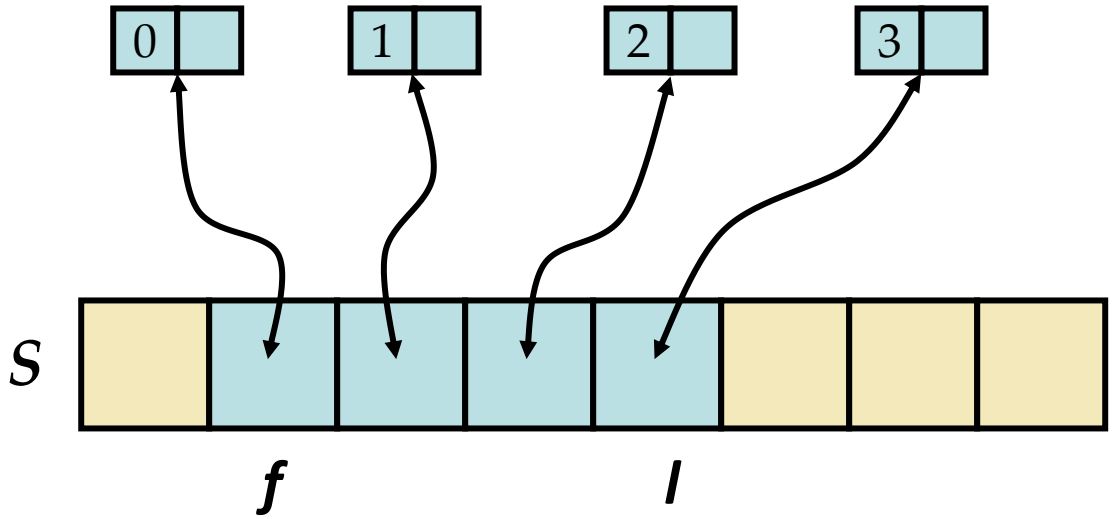
loc stores position references, not the current indices.

For one requested value x

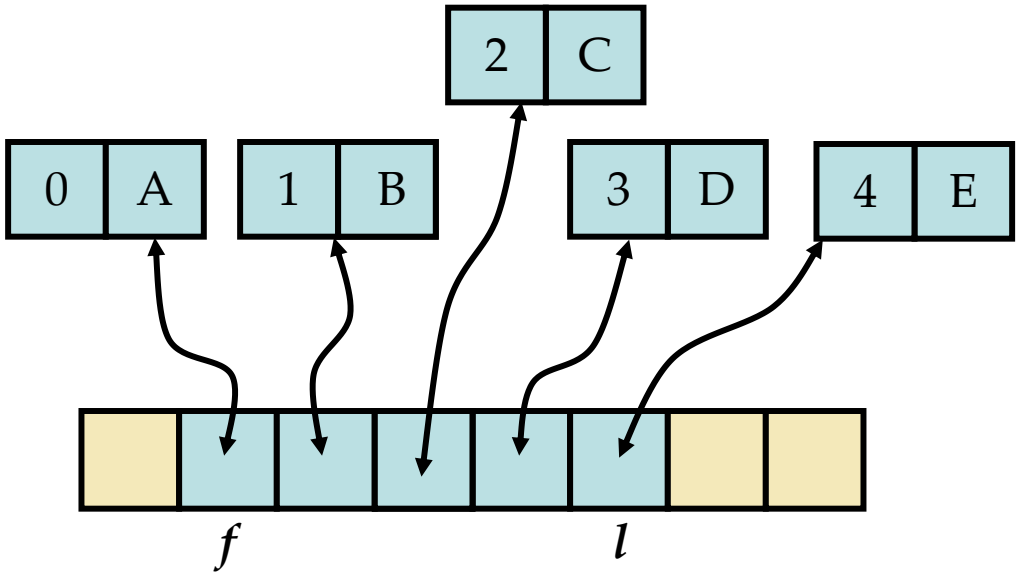
```
Algorithm MTF(x)
p ← loc[x]
r ← S.indexOf(p)
S.remove(p)
loc[x] ← S.addFirst(x)
return r
```

Implementing a sequence with an array

- A sequence consists of a circular array that stores positions
  - each position object stores: element and index
  - f and l keep track of first and last positions

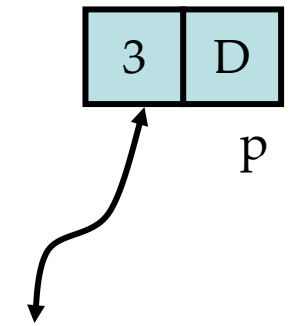


Bridge methods

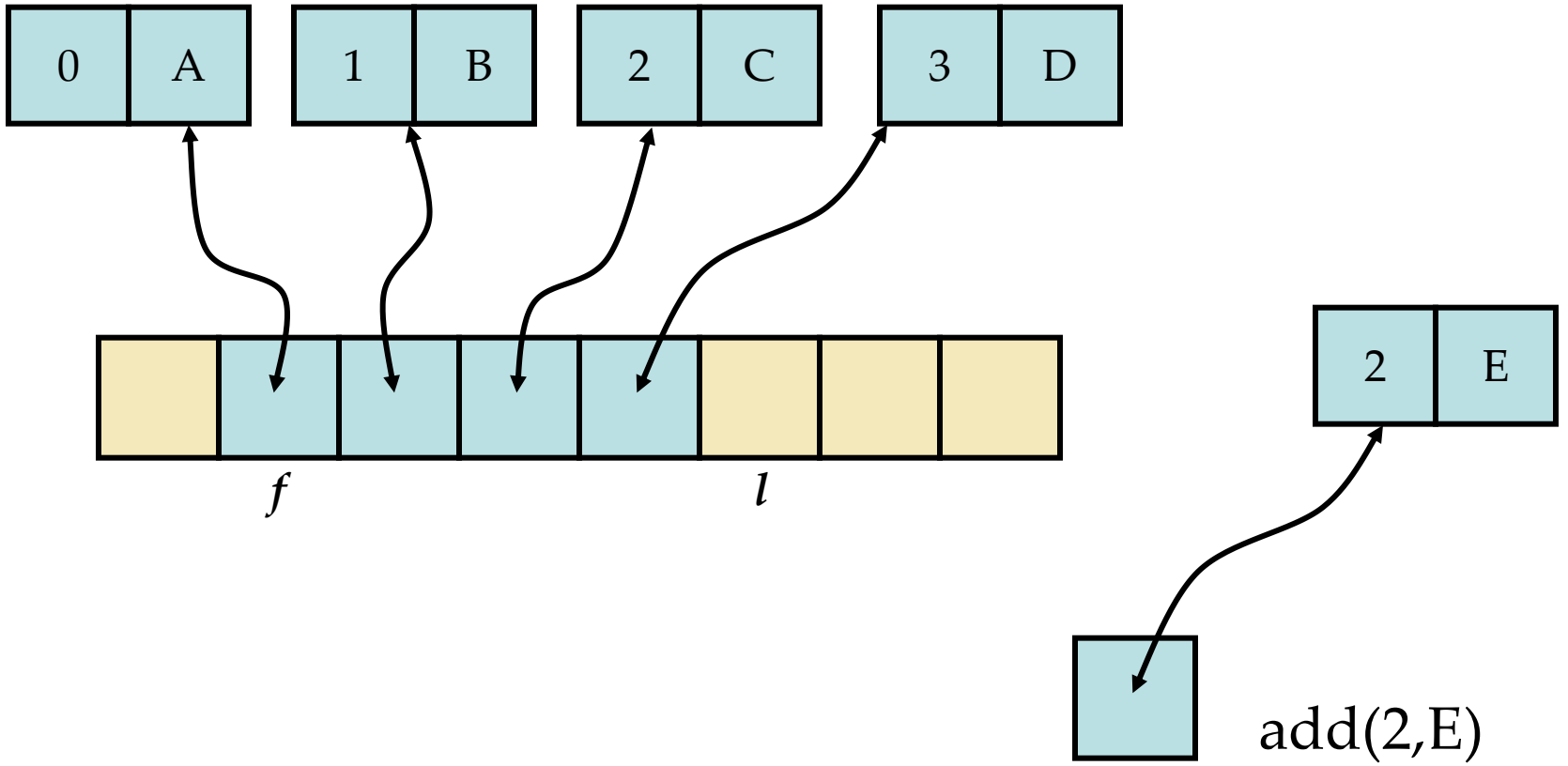


atIndex(i): Direct access to the position at index $f + i$

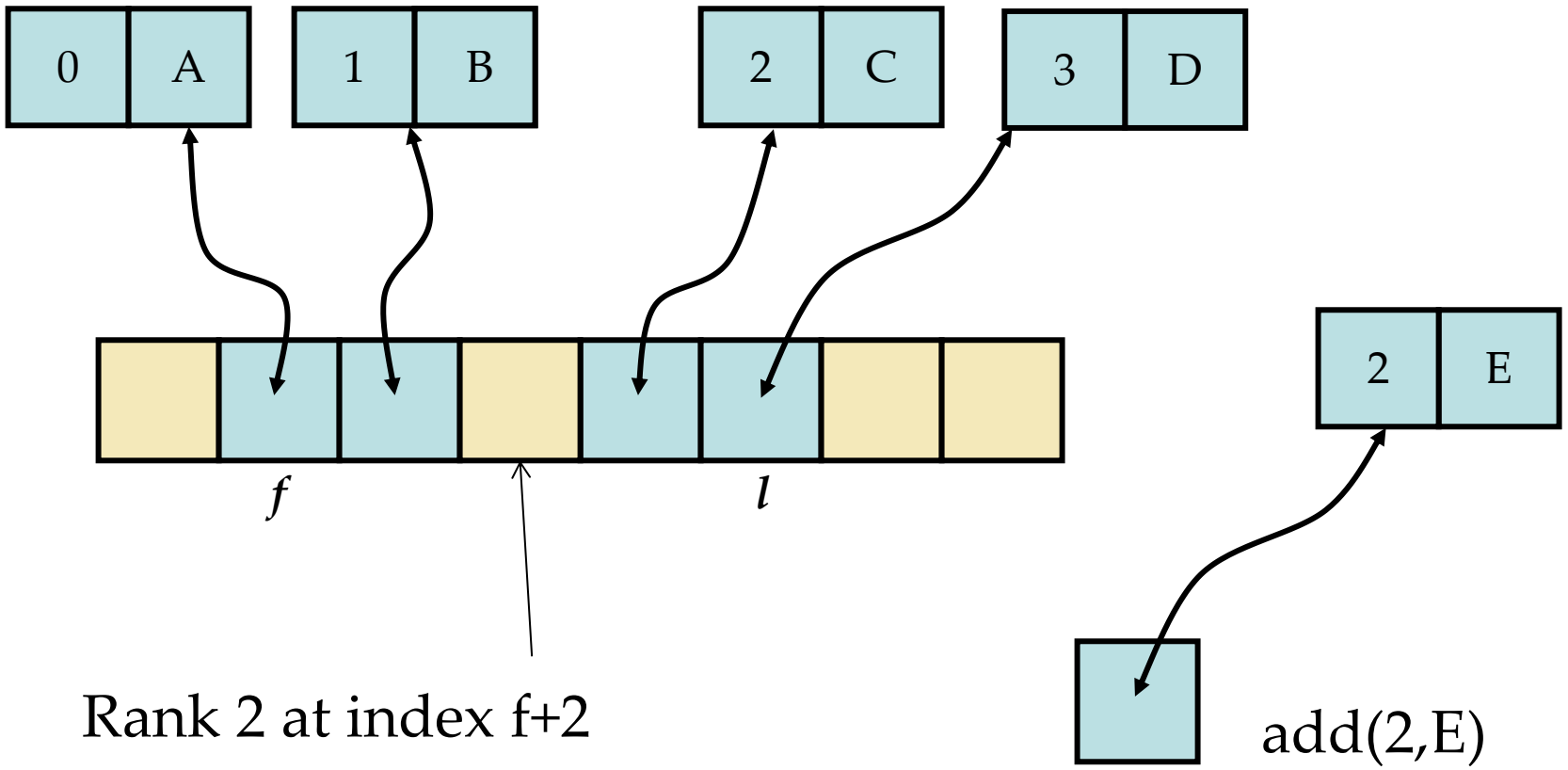
indexOf(p): Immediate access to the corresponding index



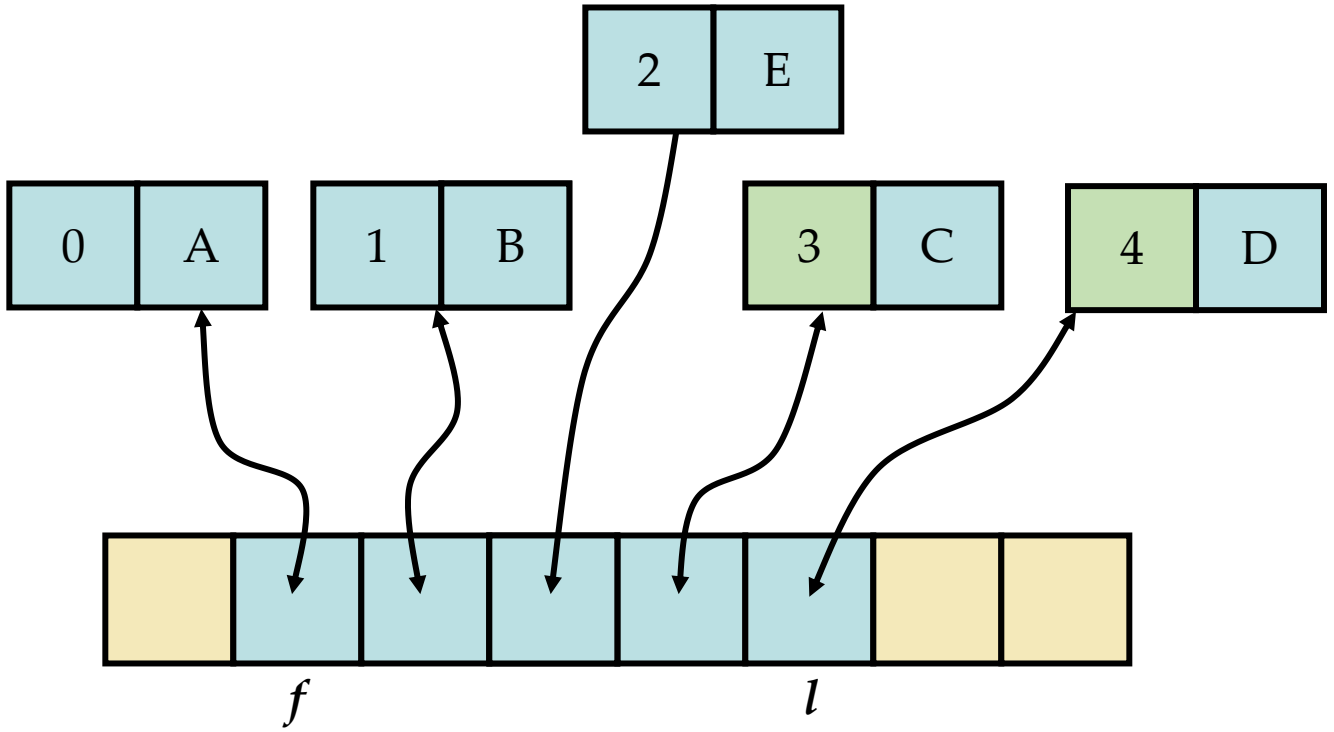
Insertion based on index



Insertion based on index



Insertion based on index



update all other ranks

Array-based sequence: Performance

Methods of array list

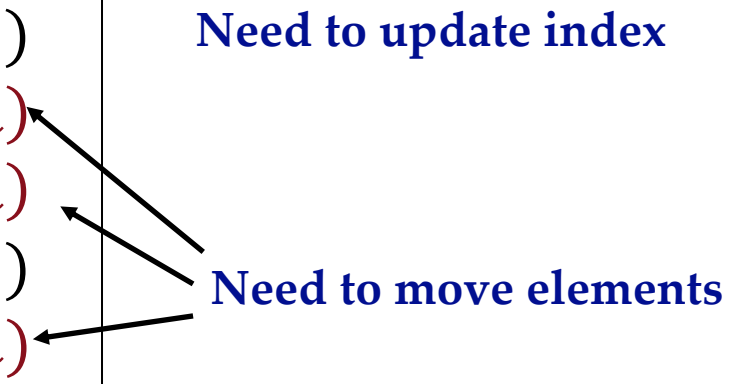
size()	$O(1)$
isEmpty()	$O(1)$
get(i)	$O(1)$
set(i,e)	$O(1)$
add(i,e)	$O(n)$
remove(i)	$O(n)$

Bridge methods

atIndex(i)	$O(1)$
indexOf(p)	$O(1)$

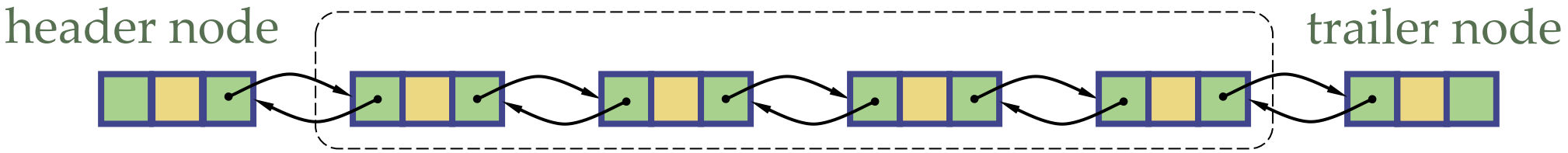
Methods of positional list

addFirst(e)	$O(n)$
addLast(e)	$O(1)$
addBefore(p,e)	$O(n)$
addAfter(p,e)	$O(n)$
set(p,e)	$O(1)$
remove(p)	$O(n)$



Implementing a sequence with linked lists

- All methods are inherited



- Bridging methods need to traverse the list

  atIndex(i): $O(n)$

  indexOf(p): $O(n)$

List-based sequence: Performance

Methods of array list

size()	$O(1)$
isEmpty()	$O(1)$
get(i)	$O(n)$
set(i,e)	$O(n)$
add(i,e)	$O(n)$
remove(i)	$O(n)$

Need to traverse to find an index

Bridging methods

atIndex(i)	$O(n)$
indexOf(p)	$O(n)$

Methods of positional list

addFirst(e)	$O(1)$
addLast(e)	$O(1)$
addBefore(p,e)	$O(1)$
addAfter(p,e)	$O(1)$
set(p,e)	$O(1)$
remove(p)	$O(1)$

Same algorithm, different sequence implementations

Move-to-front encoding performance

```
Algorithm MTF(x)
 p ← loc[x]
 i ← S.indexOf(p)
 S.remove(p)
 loc[x] ← S.addFirst(x)
 return i
```

Worst-case running time; m= number of symbols

Array-based sequence	Doubly linked sequence
O(1)	O(1)
O(1)	O(m)
O(m)	O(1)
O(m)	O(1)
O(m)	**O(m)**

Total cost per request

Array-based sequence: updates must maintain the stored ranks.

Doubly Linked sequence: finding the current rank requires traversal.

Next Topic

Trees